Chapter 1
Why one processor is no longer enough

Foundations, architecture classifications, and performance limits

01

The chapter builds one mental model in four steps

01

Recognize the need

Large problems exceed a single processor’s practical limits.

02

Choose the model

Sequential, parallel, and distributed systems organize work differently.

03

Classify the machine

Flynn’s taxonomy links instruction streams to data streams.

04

Measure the result

Speedup, efficiency, and Amdahl’s Law reveal the real gain.

More compute is needed because the problems keep growing

A single processor becomes the bottleneck when workloads demand more data, more accuracy, or faster answers.

Scale

Larger datasets and finer simulations increase the total work.

Time

Forecasting, analytics, and interactive services must finish sooner.

Energy

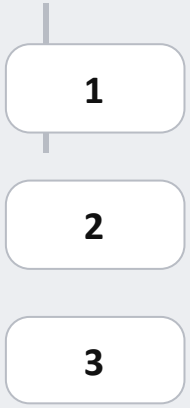
Many efficient cores can outperform one power-hungry core per watt.

AI · weather prediction · financial risk · cryptography · cloud services

The execution model determines where work happens

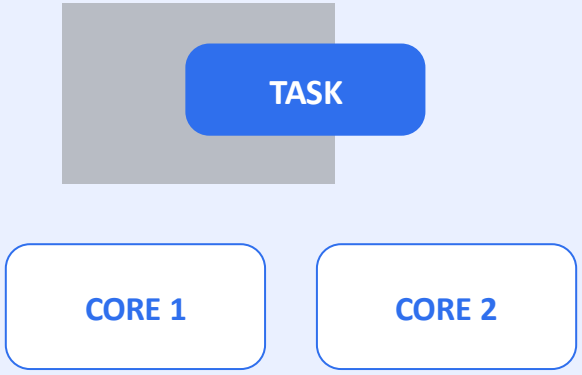
Sequential

One processing unit
One instruction after another



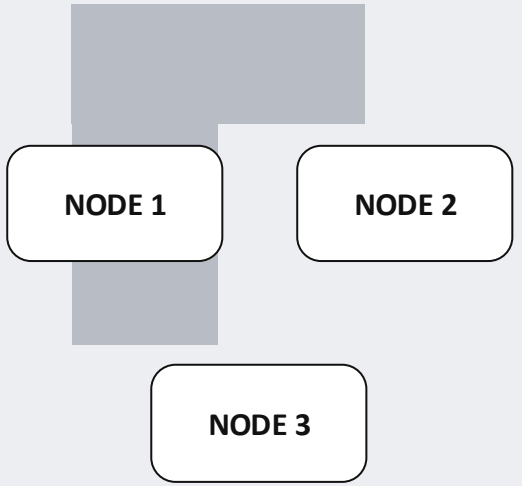
Parallel

One machine
Multiple cores work simultaneously



Distributed

Multiple machines
Own memory, connected by a network



Parallel and distributed systems coordinate differently

	Parallel computing	Distributed computing
LOCATION	Usually one computer	Multiple networked computers
MEMORY	Often shared or tightly coupled	Separate memory per node
COMMUNICATION	Shared memory or fast interconnect	Messages over a network
PRIMARY AIM	Speed up a computation	Scale capacity and coordinate services

Flynn classifies machines by instruction and data streams

	SINGLE DATA	MULTIPLE DATA
SINGLE INSTRUCTION	**SISD** — Sequential CPU One instruction handles one data item.	**SIMD** — GPU / vector unit One instruction handles many data items.
MULTIPLE INSTRUCTION	**MISD** — Rare / specialized Different instructions process the same stream.	**MIMD** — Multicore / cluster Different processors handle different work and data.

Speedup asks how much faster the parallel program runs

$S = T_{seq} \div T_{parallel}$

Compare the original sequential time with the new parallel time.

Given

Sequential time = 10 s
Parallel time = 2 s

Result

S = 10 ÷ 2 = 5×
The program runs five times faster.

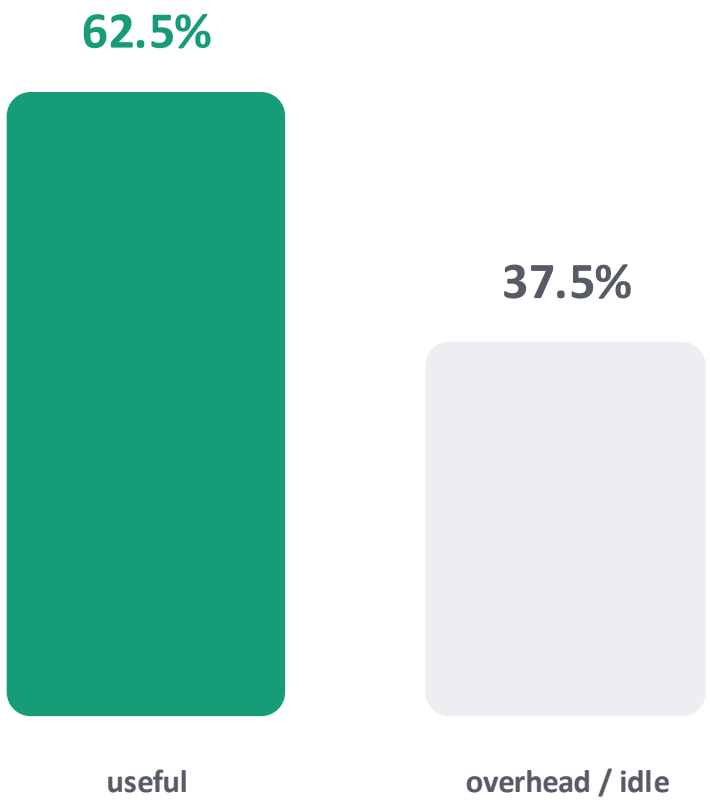
Ideal speedup with P processors is P×. Real systems usually fall below it.

Efficiency reveals useful processor capacity

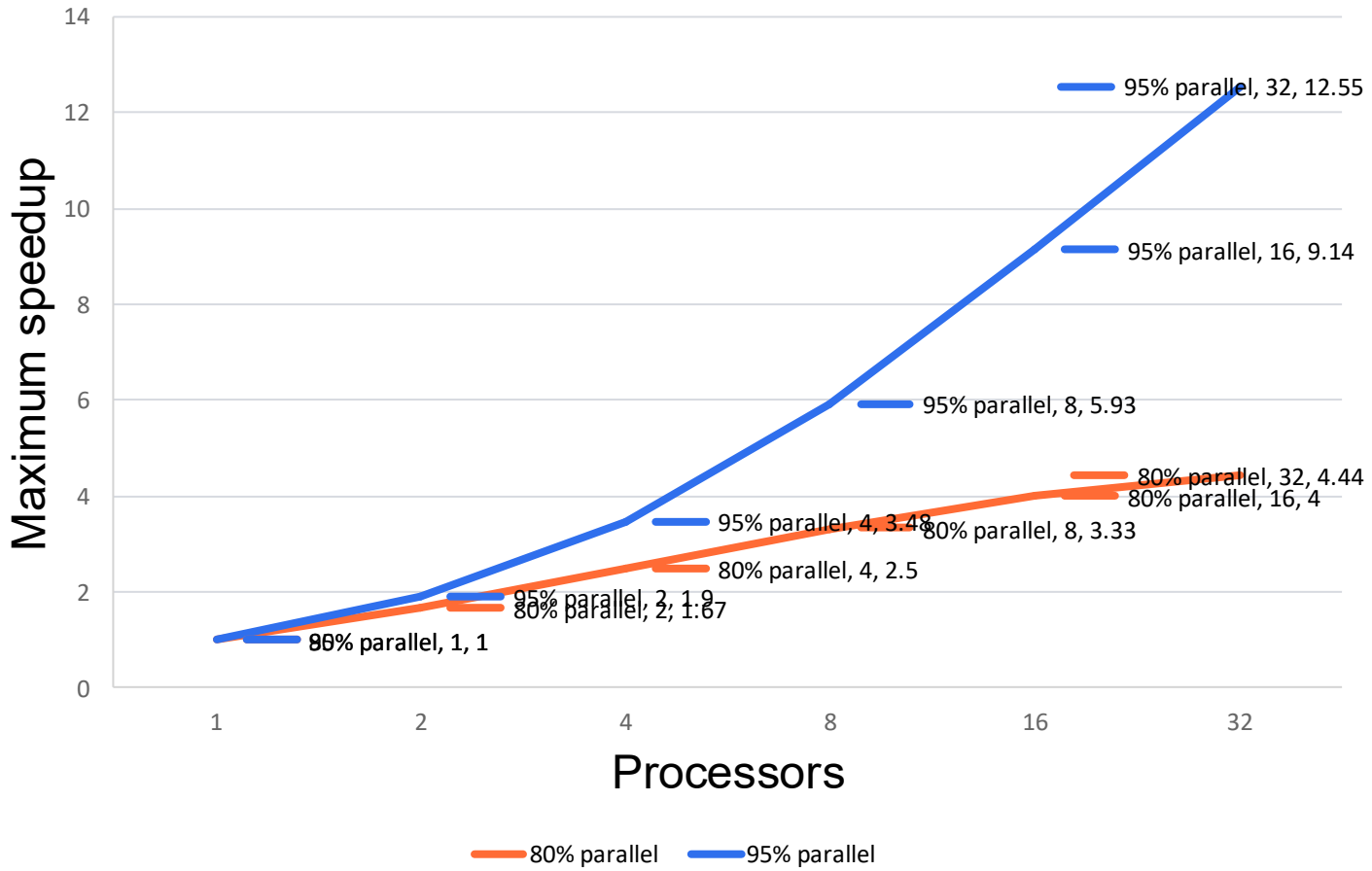
$$E = S \div P$$

With speedup S = 5 and P = 8 processors

E = 5 ÷ 8 = 0.625 = 62.5%



Amdahl’s Law makes the serial fraction impossible to ignore



$$S_{max} = 1 \div [(1 - f) + f \div P]$$

f is the parallelizable fraction
P is the processor count

The limit is structural

Adding processors cannot remove the part of the program that must run sequentially.

Worked example: 80% parallel on 8 processors

1

Identify the fractions

$f = 0.80$
$1 - f = 0.20$

2

Substitute

$S_{max} = 1 \div [0.20 + 0.80 \div 8]$

3

Simplify

$S_{max} = 1 \div 0.30 = 3.33\times$

Eight processors do not produce 8× speedup because 20% of the program remains serial.

Four practical effects pull performance below the ideal

Communication

Processors spend time moving data instead of computing.

Synchronization

Some tasks wait for others at locks, barriers, or dependencies.

Load imbalance

Uneven work leaves some processors idle while others continue.

Complexity

Concurrency makes defects harder to reproduce, test, and debug.

Check your understanding with one performance calculation

A program takes 20 seconds sequentially and 5 seconds on 4 processors.

1. Speedup

Compute $S = T_{seq} \div T_{par}$.

2. Efficiency

Compute $E = S \div P$ and convert to a percentage.

3. Interpret

What does the result say about processor utilization?

Answer: S = 4× and E = 100%

Parallel performance is a balance, not just a processor count

Model the work	Separate independent tasks from dependencies.
Match the architecture	Use Flynn’s taxonomy to reason about the machine.
Measure the outcome	Use speedup and efficiency, then explain the gap.
Respect the serial limit	Amdahl’s Law bounds the gain before overhead is added.

The best design reduces useful work time without letting communication, waiting, or serial code dominate.